

DISEÑO DE BASE DE DATOS

Manual paso a paso para aprender a modelar, diseñar y construir bases de datos relacionales

De la realidad del negocio al modelo conceptual, lógico, físico y PostgreSQL

MODELAR

NORMALIZAR

IMPLEMENTAR

Guía para el docente y material de estudio para el estudiante

Nivel: inicial → intermedio · Enfoque práctico · PostgreSQL

Cómo utilizar este manual

Este documento no enseña únicamente a “dibujar tablas”. Enseña un proceso de ingeniería: comprender un problema, descubrir datos y reglas, modelarlos sin contradicciones, transformar el modelo en relaciones correctas y finalmente implementarlo de forma segura y eficiente.

PROFESOR · Objetivo pedagógico

Al terminar, el estudiante debe ser capaz de justificar por qué existe cada tabla, cada atributo, cada clave y cada relación. El diseño correcto no se memoriza: se razona.

Etapas	Pregunta central	Producto esperado
1	¿Qué problema y procesos existen?	Alcance, actores, reglas y glosario
2	¿Qué cosas importantes debo representar?	Entidades y atributos candidatos
3	¿Cómo se relacionan?	Modelo ER y cardinalidades
4	¿Hay redundancias o dependencias problemáticas?	Modelo normalizado
5	¿Cómo lo traduzco al motor relacional?	Esquema lógico con PK, FK, UK y CHECK
6	¿Cómo lo hago robusto y eficiente?	Índices, tipos, auditoría y SQL
7	¿Cómo sé si está bien?	Pruebas con escenarios y checklist

Ruta recomendada de clases

Clase	Tema	Resultado de la clase
1	Pensar en datos	Distingue dato, información, entidad, atributo y regla
2	Levantamiento y reglas	Extrae entidades, relaciones y reglas de un caso
3	Modelo ER	Construye un DER coherente y explica cardinalidades
4	Claves y relaciones	Define PK/FK y resuelve 1:N, 1:1 y N:M
5	Normalización	Aplica 1FN, 2FN y 3FN con criterio
6	Modelo lógico	Construye tablas, restricciones y catálogo de datos
7	Modelo físico	Elige tipos, índices y convenciones PostgreSQL
8	DDL en PostgreSQL	Implementa y prueba el esquema

Clase	Tema	Resultado de la clase
9	Revisión y antipatronos	Detecta errores de modelado
10	Proyecto integrador	Defiende un diseño completo

1. Antes de modelar: aprender a pensar en datos

Una base de datos es una representación estructurada de una parte de la realidad. El diseñador decide qué hechos conservar, cómo identificarlos, cómo relacionarlos y qué reglas impedirán estados inválidos. Un buen diseño reduce ambigüedad, duplicación, pérdida de información y errores de actualización.

1.1 Conceptos fundamentales

Concepto	Definición práctica	Ejemplo
Dato	Valor elemental sin interpretación suficiente por sí solo.	"Ana", 2026-08-14, 125.50
Información	Datos organizados que responden una pregunta.	Ana compró Bs 125,50 el 14/08/2026.
Entidad	Objeto, persona, evento o concepto del que necesitamos guardar hechos.	Cliente, Producto, Pedido, Carrera
Atributo	Propiedad relevante de una entidad.	cliente.nombre, producto.precio
Relación	Asociación semántica entre entidades.	Cliente realiza Pedido
Regla de negocio	Condición que el sistema debe respetar.	Un pedido debe pertenecer a un cliente
Restricción	Mecanismo técnico que protege una regla.	NOT NULL, UNIQUE, FOREIGN KEY, CHECK

REGLA · Primer principio

No empieces por CREATE TABLE. Primero comprende la realidad que vas a representar. SQL es la traducción final, no el punto de partida.

1.2 Las cinco preguntas que un diseñador siempre debe hacer

1. ¿Qué objetos o hechos deben persistir aunque la aplicación se cierre?
2. ¿Qué identifica de manera única a cada ocurrencia?
3. ¿Qué relaciones existen y cuántas ocurrencias pueden participar en cada lado?
4. ¿Qué reglas nunca deben violarse, incluso si la aplicación se equivoca?
5. ¿Qué consultas y operaciones serán frecuentes y deben funcionar eficientemente?

1.3 Dato derivado versus dato almacenado

Un dato derivado puede calcularse a partir de otros datos. Guardarlo puede introducir inconsistencias. Por ejemplo, la edad se obtiene a partir de la fecha de nacimiento y cambia con el tiempo; normalmente conviene almacenar fecha_nacimiento, no edad.

Dato candidato	¿Almacenar?	Razón
edad	Normalmente no	Se deriva de fecha_nacimiento
subtotal de detalle	Depende	Puede derivarse de cantidad × precio; a veces se persiste por trazabilidad histórica
saldo actual	Depende	Puede calcularse, pero en sistemas transaccionales puede materializarse con controles
nombre completo	Depende	Si nombres y apellidos existen separados, puede componerse; hay contextos legales donde se conserva forma oficial

2. Del problema al modelo: levantamiento de información

Modelar comienza con lenguaje del negocio. El diseñador transforma narrativas, formularios, procesos y reglas en conceptos formales. La técnica más útil para estudiantes es leer un caso varias veces con objetivos distintos.

1 Primera lectura: comprender

Identifica el propósito del sistema, actores y procesos. No dibujes todavía.

2 Segunda lectura: subrayar sustantivos

Busca candidatos a entidad: estudiante, curso, docente, inscripción, pago.

3 Tercera lectura: subrayar verbos

Busca relaciones: estudiante se inscribe, docente dicta, cliente realiza.

4 Cuarta lectura: detectar cantidades y condiciones

Palabras como “uno”, “varios”, “obligatorio”, “opcional”, “único” sugieren cardinalidades y restricciones.

5 Quinta lectura: descubrir historia

Pregunta qué debe conservarse en el tiempo: precios aplicados, estados, cambios de carrera, responsables.

CLAVE · Caso conductor

Caso base: Una tienda registra clientes y productos. Un cliente puede realizar muchos pedidos. Cada pedido contiene uno o más productos. Para cada producto del pedido se registra cantidad y el precio aplicado al momento de la venta. Un producto puede aparecer en muchos pedidos. El correo del cliente, si se proporciona, no puede repetirse.

2.1 Extracción guiada del caso

Fragmento	Interpretación de diseño
“registra clientes y productos”	CLIENTE y PRODUCTO son entidades candidatas.
“un cliente puede realizar muchos pedidos”	Relación CLIENTE 1:N PEDIDO.
“cada pedido contiene uno o más productos”	PEDIDO se relaciona N:M con PRODUCTO.
“cantidad y precio aplicado”	Son atributos de la relación; aparece DETALLE_PEDIDO como entidad asociativa.
“correo ... no puede repetirse”	correo requiere UNIQUE; si es opcional, puede ser NULL.

TIP · Tip para descubrir entidades

Si un concepto tiene identidad propia, varios atributos, participa en relaciones o necesita conservarse históricamente, probablemente merece ser entidad.

3. Entidades y atributos: cómo decidir correctamente

3.1 ¿Entidad o atributo?

Una de las decisiones más importantes es determinar si un concepto se representa como columna o como tabla. No existe una regla mecánica universal, pero sí señales prácticas.

Pregunta	Si la respuesta es “sí”	Decisión probable
¿Tiene varios datos propios?	Dirección tiene calle, zona, ciudad, referencia.	Entidad o value object estructurado
¿Puede existir más de una por propietario?	Persona puede tener varios teléfonos.	Entidad hija / tabla separada
¿Se relaciona con otros conceptos?	Ciudad pertenece a departamento.	Entidad
¿Necesita historial?	Cargo de empleado cambia por periodos.	Entidad temporal
¿Es un valor atómico simple?	Color favorito, nombre corto.	Atributo

3.2 Atributos buenos y atributos peligrosos

Diseño débil	Problema	Mejor alternativa
telefonos = "7001,7002"	Multivalor dentro de una columna; difícil de validar y consultar.	PERSONA_TELEFONO(persona_id, numero, tipo)
direccion = "Av. X #12, SCZ"	Datos mezclados y difícil filtrado.	Separar componentes relevantes o tabla DIRECCION
producto1, producto2, producto3	Número arbitrario y repetición de columnas.	DETALLE_PEDIDO, una fila por producto
estado = texto libre	Valores inconsistentes: ACTIVO, Act, activo.	CHECK, catálogo o enum según contexto

CLAVE · Criterio de atomicidad

“Atómico” depende de lo que el sistema necesita consultar y controlar. Un nombre completo puede ser atómico para una agenda simple, pero no para un sistema académico que debe ordenar y reportar apellidos.

4. Identidad: claves primarias, naturales y sustitutas

Toda entidad persistente necesita una forma estable de distinguir una fila de otra. Una clave primaria (PK) debe ser única y no nula. En diseño moderno suele utilizarse una clave sustituta técnica y, cuando existe un identificador natural de negocio, se protege además con UNIQUE.

Tipo de clave	Ejemplo	Ventaja	Riesgo / nota
Natural	ci, email, código de producto	Tiene significado de negocio.	Puede cambiar, reutilizarse o tener excepciones.
Sustituta	id BIGINT / UUID	Estable, simple para relaciones.	No reemplaza restricciones de unicidad del negocio.
Compuesta	(pedido_id, producto_id)	Expresa identidad combinada.	Puede hacer FKs más extensas; útil en tablas asociativas.

ERROR · Error frecuente

Crear id artificial y olvidar UNIQUE en el identificador de negocio. Resultado: dos clientes con el mismo NIT, dos usuarios con el mismo correo o dos materias con el mismo código dentro del mismo contexto.

5. Relaciones y cardinalidades

La cardinalidad responde cuántas ocurrencias de una entidad pueden asociarse con una ocurrencia de la otra. Además debe distinguirse la participación mínima: cero significa opcional; uno significa obligatoria.

Relación	Lectura	Implementación típica
1 : N	Un cliente tiene muchos pedidos; cada pedido pertenece a un cliente.	FK cliente_id en PEDIDO
1 : 1	Una persona puede tener una ficha biométrica única.	FK con UNIQUE en uno de los lados
N : M	Un pedido contiene productos; un producto aparece en pedidos.	Tabla asociativa DETALLE_PEDIDO
Rekursiva	Un empleado puede depender de otro empleado.	FK jefe_id → EMPLEADO.id

5.1 Cómo leer cardinalidades con preguntas

- Para un CLIENTE concreto, ¿cuántos PEDIDOS puede haber? → 0..N.
- Para un PEDIDO concreto, ¿cuántos CLIENTES puede haber? → exactamente 1.
- ¿Puede existir un PEDIDO sin CLIENTE? Si no, cliente_id debe ser NOT NULL.
- ¿Puede un CLIENTE existir sin PEDIDOS? Si sí, no necesitas ninguna fila en PEDIDO todavía.

5.2 Resolver una relación N:M

Antes	Transformación	Después
PEDIDO N:M PRODUCTO	Crear entidad asociativa con las FKs de ambos lados y atributos propios.	PEDIDO 1:N DETALLE_PEDIDO N:1 PRODUCTO

```
pedido
- id PK
- cliente_id FK

producto
- id PK
- nombre

detalle_pedido
- pedido_id FK
- producto_id FK
- cantidad
- precio_unitario
PK posible: (pedido_id, producto_id)
```

TIP · Cuando NO usar PK compuesta en el detalle

Si un mismo producto puede aparecer más de una vez en el mismo pedido por lote, promoción, configuración u otra dimensión, (pedido_id, producto_id) ya no identifica una fila. En ese caso usa id propio o amplía la clave natural.

6. Normalización sin memorizar recetas

Normalizar significa organizar datos para representar cada hecho en el lugar correcto y evitar anomalías. No se trata de dividir tablas sin límite, sino de entender dependencias entre atributos.

Anomalía	Qué ocurre	Ejemplo
Inserción	No puedes registrar un hecho sin inventar otro.	No puedes registrar un curso hasta tener un estudiante inscrito.
Actualización	El mismo hecho debe modificarse en varias filas.	El nombre de una carrera aparece repetido 500 veces.
Eliminación	Al borrar un hecho pierdes otro distinto.	Borrar la última inscripción elimina los datos del curso.

6.1 Primera Forma Normal (1FN)

Una tabla está en 1FN cuando cada intersección fila-columna contiene un solo valor del dominio definido y no existen grupos repetitivos de columnas.

NO 1FN	1FN
cliente(id, nombre, telefonos="7001,7002")	cliente(id,nombre) + cliente_telefono(cliente_id,telefono)
pedido(id, producto1, producto2, producto3)	pedido(id,...) + detalle_pedido(pedido_id,producto_id,...)

6.2 Segunda Forma Normal (2FN)

Importa cuando existe una clave candidata compuesta. Todo atributo no clave debe depender de la clave completa, no solo de una parte.

```
MATRICULA_MATERIA(estudiante_id, materia_id, nombre_estudiante, nombre_materia, nota)
PK = (estudiante_id, materia_id)
```

```
nombre_estudiante depende solo de estudiante_id  -> mover a ESTUDIANTE
nombre_materia depende solo de materia_id        -> mover a MATERIA
nota depende de estudiante_id + materia_id        -> permanece
```

6.3 Tercera Forma Normal (3FN)

Los atributos no clave deben depender de la clave, de toda la clave y de nada más que la clave. En términos prácticos, evita dependencias transitivas entre atributos no clave.

```
EMPLEADO(id, nombre, departamento_id, departamento_nombre)
```

```
id -> departamento_id
departamento_id -> departamento_nombre
```

Solución:
 EMPLEADO(id, nombre, departamento_id FK)
 DEPARTAMENTO(id, nombre)

PROFESOR · Regla mnemotécnica útil

1FN: una celda, un valor. 2FN: depende de toda la clave. 3FN: no depende de otro atributo no clave. Úsala para estudiar, pero siempre razona las dependencias reales del negocio.

6.4 ¿Siempre hay que llegar a 3FN?

Para sistemas transaccionales, 3FN es un excelente punto de partida. La desnormalización debe ser una decisión consciente, medible y documentada, generalmente motivada por rendimiento, reporting o almacenamiento histórico; nunca por comodidad inicial.

7. Construcción del Modelo Entidad–Relación (MER/DER)

El DER debe poder explicarse verbalmente. Cada relación debe tener significado y cardinalidad. Un diagrama visualmente bonito pero semánticamente ambiguo es un mal modelo.

- 1 Lista entidades**
 Obtén una lista sin atributos todavía. Fusiona sinónimos y elimina conceptos que solo describen interfaz.
- 2 Define identidad**
 Decide PK y claves naturales que requieren UNIQUE.
- 3 Agrega atributos esenciales**
 Incluye solo datos del propio concepto; evita atributos de otras entidades.
- 4 Conecta relaciones**
 Usa verbos: cliente realiza pedido; pedido contiene producto.
- 5 Asigna cardinalidad mínima y máxima**
 Determina 0..1, 1..1, 0..N o 1..N en ambos extremos.
- 6 Resuelve N:M**
 Crea entidades asociativas y mueve allí atributos de la relación.

7

Valida con escenarios

Prueba casos normales, opcionales, duplicados e históricos.

PROFESOR · Prueba oral del modelo

Pide al estudiante que lea cada relación en ambos sentidos. Ejemplo: “Un cliente puede tener cero o muchos pedidos; cada pedido debe pertenecer a exactamente un cliente”. Si no puede leerla así, probablemente no entiende la cardinalidad.

8. Del DER al modelo relacional

El modelo lógico convierte entidades y relaciones en tablas y restricciones. Aquí ya deben aparecer columnas, tipos lógicos, PK, FK, UNIQUE, NOT NULL y CHECK, pero todavía sin obsesionarse con detalles específicos del motor.

Elemento conceptual	Traducción relacional
Entidad fuerte	Tabla
Atributo simple	Columna
Identificador	PRIMARY KEY / UNIQUE
Relación 1:N	FOREIGN KEY en el lado N
Relación 1:1	FOREIGN KEY + UNIQUE
Relación N:M	Tabla asociativa con dos FKs
Atributo obligatorio	NOT NULL
Dominio restringido	CHECK o FK a catálogo

8.1 Modelo lógico del caso de tienda

Tabla	Columnas principales	Restricciones clave
cliente	id, nombre, email	PK(id), UNIQUE(email)
producto	id, sku, nombre, precio_base, activo	PK(id), UNIQUE(sku), CHECK(precio_base >= 0)
pedido	id, cliente_id, fecha, estado	PK(id), FK(cliente_id), NOT NULL
detalle_pedido	pedido_id, producto_id, cantidad, precio_unitario	PK(pedido_id, producto_id), FKs, CHECK(cantidad > 0), CHECK(precio_unitario >= 0)

9. Tipos de datos: elegir por significado, no por costumbre

Necesidad	Tipo PostgreSQL recomendado	Observación
Identificador secuencial	BIGINT GENERATED ... AS IDENTITY	Evitar depender de SERIAL en diseños nuevos si se quiere SQL estándar moderno.
Identificador distribuido	UUID	Útil en sistemas distribuidos; evaluar patrón de generación e índices.
Texto corto	VARCHAR(n) o TEXT	VARCHAR(n) solo si el límite es una regla real.
Dinero	NUMERIC(p,s)	No usar FLOAT para importes monetarios.
Fecha sin hora	DATE	Nacimiento, fecha académica.
Instante temporal	TIMESTAMPTZ	Preferible para eventos con zona/instante real.
Booleano	BOOLEAN	Evitar S/N, 0/1 si no hay interoperabilidad que lo exija.
Documento flexible	JSONB	Solo cuando el dato realmente es semiestructurado; no sustituye buen modelado relacional.

REGLA · Dinero

NUMERIC/DECIMAL representa cantidades exactas. FLOAT/REAL son aproximados y pueden introducir errores de redondeo no aceptables en importes.

10. Restricciones: la base de datos también debe defender el negocio

Las validaciones de la interfaz son necesarias, pero no suficientes. Varias aplicaciones, integraciones o procesos batch pueden escribir en la misma base. Las reglas estructurales deben protegerse en el propio esquema cuando sea posible.

Restricción	Protege contra	Ejemplo
NOT NULL	Ausencia de dato obligatorio	pedido.cliente_id NOT NULL
UNIQUE	Duplicados lógicos	usuario.email UNIQUE
PRIMARY KEY	Duplicados de identidad y NULL	producto.id PK

Restricción	Protege contra	Ejemplo
FOREIGN KEY	Referencias inexistentes	pedido.cliente_id → cliente.id
CHECK	Valores fuera de dominio	cantidad > 0
DEFAULT	Ausencia de valor inicial conveniente	created_at DEFAULT now()

TIP • CHECK no reemplaza toda la lógica

Reglas que dependen de varias filas, agregados, tiempo o estados complejos pueden requerir transacciones, triggers cuidadosamente diseñados o lógica de dominio. No intentes forzar todo en CHECK.

11. Integridad referencial y acciones ON DELETE

Acción	Qué hace	Cuándo considerar
RESTRICT / NO ACTION	Impide eliminar si existen dependientes.	Datos maestros o transacciones donde borrar sería peligroso.
CASCADE	Elimina dependientes automáticamente.	Datos estrictamente subordinados sin vida propia, p. ej. líneas internas de una entidad efímera.
SET NULL	Conserva dependiente y rompe referencia.	Relación opcional donde perder el padre no invalida el hijo.

ERROR • No uses CASCADE por comodidad

Antes de usar ON DELETE CASCADE, describe exactamente qué cadena de filas desaparecerá. En datos académicos, financieros o históricos, el borrado físico suele ser excepcional.

12. Modelo físico e índices

El modelo físico incorpora decisiones del motor: tipos concretos, nombres, índices, tablespaces si aplica, particionamiento y estrategias de auditoría. Un índice acelera determinadas búsquedas a cambio de espacio y costo de escritura.

Patrón de consulta	Índice candidato	Comentario
Buscar usuario por email	UNIQUE(email)	Además garantiza unicidad.
Listar pedidos de un cliente por fecha	INDEX(cliente_id, fecha DESC)	El orden de columnas debe seguir el patrón de filtrado/ordenamiento.

Patrón de consulta	Índice candidato	Comentario
Uniones por FK frecuente	INDEX(FK)	PostgreSQL no crea automáticamente índice en la columna FK hija.
Filtrar por estado con pocos valores	Depende	Un índice simple puede aportar poco; medir selectividad y volumen.

REGLA · Regla de oro de índices

No indexes “todas las columnas”. Diseña índices a partir de consultas reales y confirma su utilidad con EXPLAIN/EXPLAIN ANALYZE.

13. Convenciones de nombres que evitan caos

- Usa una convención única: snake_case es natural en PostgreSQL.
- Usa nombres claros y completos: fecha_nacimiento mejor que fnac si no existe estándar institucional.
- Evita palabras reservadas como user, order o group sin estrategia clara.
- FKs con patrón <entidad>_id: cliente_id, pedido_id.
- Campos temporales consistentes: created_at, updated_at; y, si aplica, created_by, updated_by.
- No mezcles español e inglés dentro del mismo modelo salvo una decisión institucional explícita.

Evitar	Preferir
tbl_cli	cliente
cod	codigo_producto o codigo según contexto
fecha	fecha_pedido / fecha_nacimiento
status	estado si todo el modelo está en español
idCliente	cliente_id

14. Implementación completa en PostgreSQL

El siguiente DDL implementa el caso conductor. El objetivo del estudiante debe ser poder explicar cada línea, no copiarla.

```
CREATE TABLE cliente (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre      VARCHAR(150) NOT NULL,
  email       VARCHAR(254),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uk_cliente_email UNIQUE (email)
);
```

```

CREATE TABLE producto (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  sku         VARCHAR(50) NOT NULL,
  nombre      VARCHAR(180) NOT NULL,
  precio_base NUMERIC(12,2) NOT NULL,
  activo      BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT uk_producto_sku UNIQUE (sku),
  CONSTRAINT ck_producto_precio CHECK (precio_base >= 0)
);

CREATE TABLE pedido (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  cliente_id  BIGINT NOT NULL,
  fecha       TIMESTAMPTZ NOT NULL DEFAULT now(),
  estado      VARCHAR(20) NOT NULL DEFAULT 'CREADO',
  CONSTRAINT fk_pedido_cliente
    FOREIGN KEY (cliente_id) REFERENCES cliente(id),
  CONSTRAINT ck_pedido_estado
    CHECK (estado IN ('CREADO','CONFIRMADO','CANCELADO'))
);

CREATE TABLE detalle_pedido (
  pedido_id   BIGINT NOT NULL,
  producto_id BIGINT NOT NULL,
  cantidad    INTEGER NOT NULL,
  precio_unitario NUMERIC(12,2) NOT NULL,
  CONSTRAINT pk_detalle_pedido PRIMARY KEY (pedido_id, producto_id),
  CONSTRAINT fk_detalle_pedido_pedido
    FOREIGN KEY (pedido_id) REFERENCES pedido(id) ON DELETE CASCADE,
  CONSTRAINT fk_detalle_pedido_producto
    FOREIGN KEY (producto_id) REFERENCES producto(id),
  CONSTRAINT ck_detalle_cantidad CHECK (cantidad > 0),
  CONSTRAINT ck_detalle_precio CHECK (precio_unitario >= 0)
);

CREATE INDEX ix_pedido_cliente_fecha
  ON pedido (cliente_id, fecha DESC);

CREATE INDEX ix_detalle_producto
  ON detalle_pedido (producto_id);

```

14.1 ¿Por qué precio_unitario está en detalle_pedido?

Porque el precio_base del producto puede cambiar mañana. El pedido necesita conservar el precio efectivamente pactado en el momento de la transacción. Es un ejemplo de dato histórico legítimamente persistido, aunque parezca derivable del producto actual.

15. Cómo probar un diseño antes de programar la aplicación

Un modelo se valida con escenarios. No basta con que el DDL compile. Debe aceptar situaciones válidas y rechazar situaciones inválidas.

Prueba	Resultado esperado	Regla validada
Crear cliente sin email	Acepta	email es opcional
Crear dos clientes con mismo email no nulo	Rechaza	UNIQUE(email)

Prueba	Resultado esperado	Regla validada
Crear pedido con cliente inexistente	Rechaza	FOREIGN KEY
Agregar detalle con cantidad 0	Rechaza	CHECK cantidad > 0
Eliminar producto usado por detalle	Rechaza por defecto	Integridad histórica
Eliminar pedido	Elimina sus detalles si se decidió CASCADE	Dependencia detalle→pedido

16. Errores clásicos de estudiantes y cómo corregirlos

Error	Por qué falla	Corrección
Una tabla gigante para todo	Mezcla hechos distintos, genera NULLs y redundancia.	Separar entidades y relaciones según dependencias.
Una tabla por pantalla	La UI no define el dominio. Una pantalla puede combinar muchas entidades.	Modelar primero el negocio.
Guardar listas en VARCHAR	Rompe atomicidad y dificulta búsquedas/restricciones.	Crear tabla hija o asociativa.
N:M directa sin tabla intermedia	No puede representarse correctamente en el modelo relacional.	Crear entidad asociativa.
Todo nullable	La base acepta estados incompletos.	Definir obligatoriedad real con NOT NULL.
Todo CASCADE	Puede provocar borrados masivos no deseados.	Elegir política por semántica.
Usar nombre como PK	Nombres cambian y se repiten.	PK estable + UNIQUE cuando corresponda.
No guardar historia	Cambios posteriores alteran el significado de transacciones pasadas.	Persistir snapshots necesarios o modelar vigencia.
Índice en cada columna	Costo de escritura y almacenamiento sin beneficio claro.	Indexar según consultas.
Confiar solo en frontend	La integridad puede romperse desde otras vías.	Restricciones en BD + lógica de aplicación.

17. Diccionario de datos: documentar lo que construimos

El diccionario de datos convierte el esquema en documentación verificable. Cada columna debe tener propósito, tipo, obligatoriedad, reglas y significado.

Campo	Tipo	Nulo	Clave	Regla / significado
id	BIGINT	No	PK	Identificador técnico del cliente.
nombre	VARCHAR(150)	No	—	Nombre de presentación del cliente.
email	VARCHAR(254)	Sí	UK	Correo; si existe, no puede repetirse.
created_at	TIMESTAMPZ	No	—	Instante de creación; default now().

PROFESOR • Evaluación sugerida

No aceptar un DER sin diccionario de datos. El estudiante debe demostrar que entiende el significado y reglas de cada atributo, no solo su nombre.

18. Auditoría, estados e historia

Muchos sistemas reales no pueden limitarse al estado actual. Deben responder quién creó o modificó un registro, cuándo ocurrió y qué estado tenía una entidad en determinado momento.

Necesidad	Patrón posible
Auditoría básica	created_at, updated_at, created_by, updated_by
Borrado lógico	deleted_at o activo; usar solo cuando el negocio lo requiere
Estados	campo estado + reglas de transición en dominio; historial si importa conocer cambios
Vigencia	valid_from / valid_to o tablas históricas
Trazabilidad transaccional	eventos/outbox/auditoría separada según criticidad

TIP • No agregues deleted_at automáticamente

El borrado lógico complica unicidad, consultas e índices. Debe responder a una necesidad de negocio o auditoría concreta.

19. Patrones de modelado útiles

Patrón	Cuándo aparece	Modelo típico
Cabecera-detalle	Facturas, pedidos, matrículas con ítems	pedido + detalle_pedido
Catálogo	Valores administrados con metadatos	tipo_documento, ciudad, categoría
Jerarquía	Categorías o unidades con padre	categoria(parent_id FK a categoria)
Vigencia	Asignaciones que cambian en el tiempo	persona_cargo(persona_id,cargo_id,d esde,hasta)
Entidad asociativa con atributos	N:M con datos propios	inscripcion(estudiante_id,curso_id,fe cha,estado)
Snapshot histórico	El valor debe quedar congelado	detalle.precio_unitario, factura.razon_social_snapshot

20. ¿CHECK, ENUM o tabla catálogo?

Opción	Úsala cuando	Ejemplo
CHECK	Conjunto pequeño, estable y propio de una tabla.	cantidad > 0; estado IN (...) si muy estable
ENUM PostgreSQL	Conjunto estable y semánticamente fuerte; aceptar acoplamiento al motor.	Tipo técnico muy estable
Tabla catálogo	Valores administrables, con código, nombre, vigencia, orden, traducción o relaciones.	tipo_documento, carrera, categoría

CLAVE · Pregunta decisiva

¿El negocio necesita administrar esos valores como datos? Si la respuesta es sí, probablemente es una tabla catálogo.

21. Método completo de diseño: del enunciado al DDL

#	Actividad	Evidencia
1	Definir alcance	Qué proceso cubre y qué queda fuera.
2	Crear glosario	Unificar términos y evitar sinónimos ambiguos.
3	Identificar entidades	Sustantivos con identidad o hechos persistentes.

#	Actividad	Evidencia
4	Identificar atributos	Propiedades propias, atómicas según uso.
5	Definir claves	PK estable + claves naturales protegidas.
6	Descubrir relaciones	Verbos y dependencias semánticas.
7	Asignar cardinalidades	Mínimo y máximo en ambos sentidos.
8	Resolver N:M	Entidades asociativas y atributos de relación.
9	Normalizar	1FN→2FN→3FN mediante dependencias.
10	Definir restricciones	NOT NULL, UNIQUE, FK, CHECK.
11	Diseñar historia/auditoría	Snapshots, vigencia y estados cuando apliquen.
12	Elegir tipos físicos	PostgreSQL: NUMERIC, TIMESTAMPTZ, UUID/BIGINT, etc.
13	Diseñar índices	A partir de consultas y FKs frecuentes.
14	Construir DDL	Orden correcto de tablas y constraints.
15	Sembrar datos de prueba	Casos normales y casos frontera.
16	Validar consultas	JOINS y filtros clave del negocio.
17	Revisar con checklist	Semántica, integridad, claridad y rendimiento.

22. Ejemplo guiado 2: sistema académico básico

Enunciado: Una universidad registra estudiantes y asignaturas. Cada periodo académico ofrece grupos de una asignatura. Un docente dicta cada grupo. Los estudiantes se inscriben a grupos y la inscripción conserva fecha, estado y nota final cuando corresponda.

22.1 Entidades candidatas

- ESTUDIANTE
- ASIGNATURA
- PERIODO_ACADEMICO
- DOCENTE
- GRUPO_OFERTADO
- INSCRIPCION

22.2 Relaciones

Relación	Cardinalidad sugerida	Observación
ASIGNATURA → GRUPO_OFERTADO	1:N	La asignatura puede ofrecerse muchas veces por periodo/sección.
PERIODO → GRUPO_OFERTADO	1:N	Cada grupo pertenece a un periodo.
DOCENTE → GRUPO_OFERTADO	1:N	Simplificación: un docente responsable por grupo.
ESTUDIANTE ↔ GRUPO_OFERTADO	N:M resuelta por INSCRIPCION	Inscripción tiene atributos propios.

22.3 Restricciones que el estudiante debe descubrir

- No debe existir dos veces la misma inscripción del mismo estudiante al mismo grupo: UNIQUE(estudiante_id, grupo_id).
- nota_final debe respetar el rango institucional, por ejemplo 0..100, y puede ser NULL mientras no se cierre la evaluación.
- grupo_ofertado debe referenciar una asignatura, un periodo y un docente válidos.
- El código de asignatura debe ser único dentro del contexto definido por la institución.
- No conviene repetir nombre_asignatura dentro de INSCRIPCION.

23. Actividades de aula paso a paso

Actividad 1 · Subrayar el dominio

Entregar un enunciado de una clínica. En parejas: marcar sustantivos, verbos, reglas y cantidades. Producto: cuatro listas.

Actividad 2 · Entidad o atributo

Dar 20 conceptos: teléfono, ciudad, pedido, color, dirección, carrera, cuota, estado. Cada grupo justifica su decisión.

Actividad 3 · Cardinalidades orales

El docente pronuncia relaciones y el estudiante debe leerlas en ambos sentidos con mínimo y máximo.

Actividad 4 · Cazar anomalías

Entregar una tabla desnormalizada y pedir ejemplos de anomalía de inserción, actualización y eliminación.

Actividad 5 · Normalizar

Transformar una tabla de matrículas hasta 3FN mostrando dependencias.

Actividad 6 · DDL defensivo

Implementar el modelo y construir INSERTs que demuestren que las restricciones rechazan datos inválidos.

Actividad 7 · Revisión cruzada

Cada grupo revisa el DER de otro grupo usando el checklist del capítulo 25.

24. Banco de ejercicios para estudiantes

Caso	Alcance mínimo
Biblioteca	Libros, autores, ejemplares físicos, lectores y préstamos. Un libro puede tener varios autores y varios ejemplares.
Veterinaria	Mascotas, propietarios, veterinarios, consultas, diagnósticos y vacunas.
Restaurante	Mesas, clientes, pedidos, productos del menú, detalle, mesero y pagos.
Taller mecánico	Vehículos, propietarios, órdenes de trabajo, servicios, repuestos y técnicos.
Condominio	Propietarios, unidades, residentes, cuotas, pagos, reservas de áreas comunes.
Cursos online	Cursos, módulos, lecciones, instructores, estudiantes, inscripciones y progreso.
Hotel	Huéspedes, habitaciones, tipos, reservas, consumos y pagos.
Marketplace	Usuarios, vendedores, productos, publicaciones, pedidos, detalle, envíos y reseñas.

Consigna común

10. Escribir alcance y al menos 10 reglas de negocio.
11. Identificar entidades, atributos y claves.
12. Construir DER con cardinalidades mínimas y máximas.
13. Resolver N:M.
14. Normalizar hasta 3FN y justificar.
15. Crear diccionario de datos.
16. Implementar DDL en PostgreSQL.
17. Crear al menos 15 pruebas de integridad, incluyendo errores esperados.
18. Proponer 3 consultas frecuentes y sus índices candidatos.
19. Defender oralmente decisiones de diseño.

25. Checklist brutal de revisión de un modelo

Área	Pregunta de control
Semántica	¿Cada tabla representa un solo tipo de hecho o concepto?
Semántica	¿Cada nombre puede explicarse sin mirar la interfaz?

Área	Pregunta de control
Identidad	¿Toda entidad tiene PK estable?
Identidad	¿Las claves naturales importantes están protegidas con UNIQUE?
Atributos	¿No existen listas separadas por comas ni grupos repetitivos?
Atributos	¿Cada atributo pertenece realmente a esa entidad?
Relaciones	¿Toda relación tiene cardinalidad mínima y máxima?
Relaciones	¿Toda N:M está resuelta correctamente?
Integridad	¿Las FKs representan relaciones reales y tienen nullability correcta?
Integridad	¿Los obligatorios son NOT NULL?
Integridad	¿Rangos y dominios simples tienen CHECK?
Normalización	¿Hay dependencias parciales o transitivas evitables?
Historia	¿Los datos históricos que deben congelarse se conservan?
Borrado	¿ON DELETE fue decidido conscientemente?
Tipos	¿Los importes usan NUMERIC y los instantes TIMESTAMPTZ cuando corresponde?
Índices	¿Los índices responden a consultas y relaciones reales?
Nombres	¿La convención es consistente y legible?
Documentación	¿Existe diccionario de datos y reglas?
Pruebas	¿Se han probado estados inválidos y duplicados?
Defensa	¿El estudiante puede justificar cada decisión?

26. Rúbrica sugerida para evaluar diseño de base de datos

Criterio	Peso	Excelente	Deficiente
Comprensión del dominio	15%	Alcance, actores y reglas claros.	Tablas inventadas desde la UI sin reglas.
Modelo conceptual	20%	Entidades y cardinalidades correctas y justificadas.	Relaciones ambiguas o N:M sin resolver.

Criterio	Peso	Excelente	Deficiente
Normalización	15%	Dependencias explicadas y 3FN razonable.	Redundancia y anomalías evidentes.
Modelo lógico e integridad	20%	PK/FK/UK/NOT NULL/CHECK coherentes.	Integridad confiada solo a la aplicación.
Modelo físico	10%	Tipos e índices adecuados al uso.	Tipos genéricos e índices sin criterio.
Documentación y pruebas	10%	Diccionario y pruebas positivas/negativas.	Sin documentación verificable.
Defensa individual	10%	Justifica decisiones y responde escenarios.	No comprende el modelo construido.

27. Tips del profesor para enseñar modelado

- No corrijas primero la notación gráfica; corrige primero el significado.
- Pregunta “¿qué pasa si...?": si un cliente no tiene pedidos, si cambia el precio, si se repite un correo, si se elimina un padre.
- Exige lectura verbal de cardinalidades en ambos sentidos.
- Pide ejemplos de filas reales; muchas incoherencias aparecen al intentar insertar datos concretos.
- Antes de normalizar, pide que el estudiante diga cuál es el hecho representado por cada fila.
- No premies modelos con demasiadas tablas por parecer “más profesionales”. Premia claridad semántica y necesidad real.
- Usa errores de INSERT como evidencia pedagógica: una buena constraint debe impedir el dato inválido.
- En defensa, cambia una regla del negocio y pregunta qué partes del modelo se ven afectadas.
- Haz que comparen dos diseños válidos y discutan trade-offs: PK compuesta vs sustituta, CHECK vs catálogo, snapshot vs derivación.

28. Proyecto integrador de diseño

Proyecto: Sistema de reservas y atención de un centro deportivo. El objetivo es aplicar el proceso completo sin comenzar por código.

- Gestionar clientes, entrenadores, disciplinas, espacios, horarios, reservas y asistencia.
- Un entrenador puede estar habilitado para varias disciplinas.
- Una disciplina puede ser impartida por varios entrenadores.
- Los espacios tienen capacidad y pueden soportar determinadas disciplinas.
- Una reserva corresponde a un cliente, una sesión programada y un estado.
- No se debe duplicar una reserva activa del mismo cliente para la misma sesión.
- La sesión debe conservar horario, entrenador, disciplina y espacio.
- Debe poder registrarse cancelación y asistencia sin destruir la historia.

Entregables del estudiante

Entrega	Contenido
A	Alcance, glosario, actores, 20 reglas de negocio, supuestos y exclusiones.
B	DER conceptual con cardinalidades y explicación de cada relación.
C	Normalización y modelo lógico con diccionario de datos.
D	DDL PostgreSQL, constraints, índices y datos semilla.
E	20 pruebas de integridad, 5 consultas de negocio y defensa oral.

29. Modelo mental final del diseñador

CLAVE · Secuencia que el estudiante debe internalizar

REALIDAD → REGLAS → HECHOS → ENTIDADES → IDENTIDAD → RELACIONES →
CARDINALIDADES → DEPENDENCIAS → NORMALIZACIÓN → RESTRICCIONES → TIPOS → ÍNDICES
→ SQL → PRUEBAS.

El diseño de base de datos es una disciplina de precisión semántica. La calidad del modelo no se mide por la cantidad de tablas ni por la complejidad del diagrama, sino por su capacidad de representar los hechos correctos, impedir contradicciones, conservar la historia necesaria y responder eficientemente a las operaciones del sistema.

30. Hoja rápida de referencia

Si ves...	Piensa en...
“muchos” en ambos sentidos	N:M → tabla asociativa
dato que se repite en muchas filas	posible entidad separada / normalización
lista dentro de una celda	violación de 1FN / tabla hija
atributo que depende solo de parte de PK compuesta	violación de 2FN
atributo no clave que determina otro no clave	violación de 3FN
identificador de negocio	UNIQUE además de PK técnica
valor que cambia pero debe conservarse históricamente	snapshot o modelo temporal
dato obligatorio	NOT NULL

Si ves...	Piensa en...
rango o conjunto pequeño estable	CHECK
búsqueda frecuente por columnas específicas	índice candidato
FK usada en joins	considerar índice en columna hija

FIN DEL MANUAL

El objetivo no es que el estudiante memorice tablas: es que aprenda a justificar modelos.